

SSO Unila Upgrade — Brainstorm & Perbandingan Opsi

Inisiatif: Modernisasi SSO Unila + Integrasi RADIUS WiFi Internal **Status:** Brainstorm v2 — diskusi lanjutan dengan tambahan analisis Entra ID, MFA migration, dan tahapan migrasi aman **Tanggal:** 13 Mei 2026 **Owner:** UPA TIK Universitas Lampung **Repo terkait:** `my-unila/` , `library` `muhammadikhsan94/sso-unila` (composer `unila/sso`)

Perubahan v2 vs v1

- Tambah **Bagian 4.5:** Referensi implementasi SSO ITS Surabaya & IPB
 - Tambah **Bagian 4.6:** Opsi D — Microsoft Entra ID sebagai primary (dan kenapa **bukan rekomendasi**)
 - Tambah **Bagian 4.7:** Federation Pattern (Keycloak ↔ Entra ↔ Google) — pola yang dipakai
 - Tambah **Bagian 8.5:** MFA — Migrasi dari MyUnila Existing ke Keycloak
 - Tambah **Bagian 12 (KUNCI):** Tahapan Migrasi Aman (User-Affecting Phases) — 9 fase rolling
 - Tambah **Appendix A:** Protocol Glossary (CAS / SAML / OIDC / LDAP / RADIUS / SCIM)
-

Daftar Isi

1. [Executive Summary](#)
 2. [Kondisi Sekarang \(Current State\)](#)
 3. [Tiga Opsi Upgrade](#)
 4. [Tabel Perbandingan Lengkap \(Current vs A vs B vs C vs D\)](#)
 5. [Perbandingan Aspek Keamanan](#)
 6. [Integrasi RADIUS WiFi per Opsi](#)
 7. [Arsitektur Database \(Pisah vs Shared\)](#)
 8. [Strategi Migrasi Password Lama & MFA](#)
 9. [Estimasi Biaya & Effort per Opsi](#)
 10. [Rekomendasi Final & Justifikasi](#)
 11. [Open Questions](#)
 12. [Tahapan Migrasi Aman \(User-Affecting Phases\)](#)
 13. [Appendix A: Protocol Glossary](#)
-

1. Executive Summary

SSO Unila saat ini berbasis **CAS (Central Authentication Service)** dengan client library PHP (`composer require unila/sso`) yang dipakai aplikasi internal Unila. Sistem ini punya beberapa keterbatasan kritis:

- **Protokol legacy** — hanya CAS, tidak support OAuth2/OIDC/SAML untuk aplikasi modern.
- **Password hash lemah** — SHA1(YYYYMMDD+NPM), deprecated > 15 tahun untuk credential storage.
- **Tidak ada MFA** dan tidak ada audit log terstandar.
- **Tightly coupled** — login MyUnila man-akses query langsung ke PDUT (DB produksi shared).
- **RADIUS MySQL** terpisah dengan format password non-standar.

Dokumen ini membandingkan **3 opsi upgrade**:

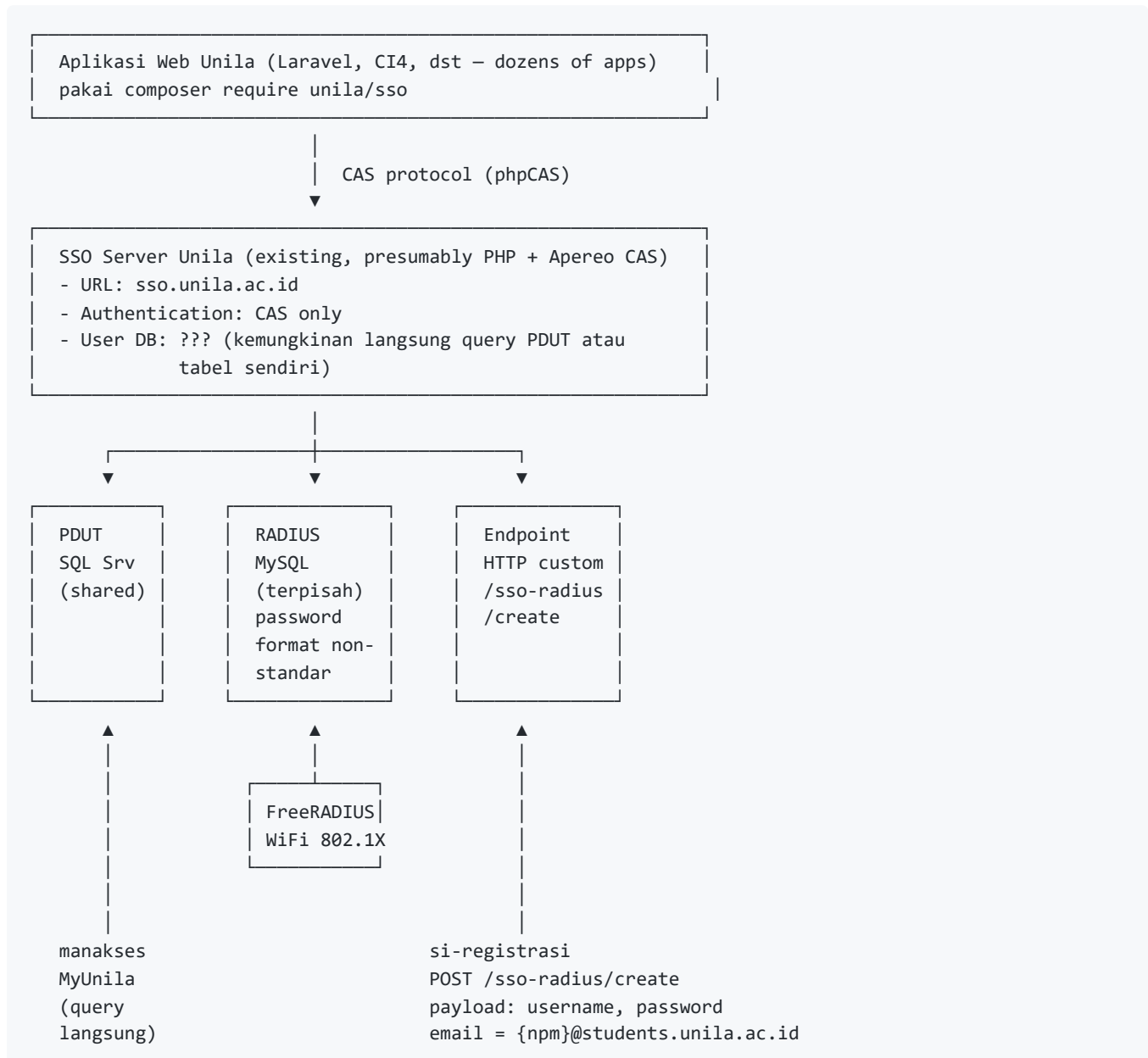
- **Opsi A: Apereo CAS Java Server** — incumbent path, paling konservatif, native CAS.
- **Opsi B: Keycloak** — open-source IdP modern, satu stop untuk OIDC+SAML+CAS+LDAP+MFA.
- **Opsi C: Custom Go IdP** — bangun sendiri dengan stack Unila eksisting.

Plus **Integrasi RADIUS** untuk WiFi internal kampus (EAP-TTLS-PAP, EAP-PEAP-MSCHAPv2, EAP-TLS).

Rekomendasi: Opsi B (Keycloak) + FreeRADIUS dengan EAP-TTLS-PAP ke Keycloak via REST.

2. Kondisi Sekarang (Current State)

2.1 Komponen Eksisting



2.2 Atribut yang Dikembalikan Library `unila/sso`

```
$user = SSO::getUser();  
// stdClass {  
//   username: "2617051073"  
//   email: "2617051073@students.unila.ac.id"  
//   nm_pengguna: "Nama Lengkap"  
//   a_aktif: true  
// }
```

Bukan klaim OIDC standar. Aplikasi konsumen harus hard-code field Unila-specific.

2.3 Aliran Saat Login

Aktor	Aksi
User	Buka aplikasi internal (mis. <code>manakses.myunila.unila.ac.id</code>)
Aplikasi	<code>SSO::authenticate()</code> → redirect ke <code>sso.unila.ac.id/login</code>
User	Input NPM/NIP + password di form CAS
SSO Server	Query DB user (PDUT atau internal), verify hash (kemungkinan SHA1)
SSO Server	Issue CAS service ticket → redirect ke aplikasi
Aplikasi	<code>phpCAS::validate()</code> → ambil atribut, simpan ke session PHP
User	Lanjut akses aplikasi

2.4 Aliran Provisioning User Baru (dari si-registrasi)

Aktor	Aksi
si-registrasi	Setelah NPM generated, call <code>SsoService::createUser(\$data)</code>
SsoService	Build JWT self-signed (HMAC-SHA256, 11 claim)
SsoService	<code>POST {sso}/sso-radius/create</code> dengan payload <code>{username, value, email, ...}</code>
	<code>value = SHA1(YYYYMMDD + NPM)</code> → dipakai sebagai password
	<code>email = {npm}@students.unila.ac.id</code> (hardcoded)
SSO Server	Insert user ke DB + RADIUS MySQL

2.5 Risiko Spesifik

#	Risiko	Severity
C1	MD5 tanpa salt di RADIUS MySQL (terkonfirmasi) — rainbow table publik, brute-force <30 detik untuk password lemah	Critical
C2	SHA1(YYYYMMDD+NPM) di SSO Unila — deterministic; tgl lahir + NPM publik = password ter-tebak	Critical
C3	Tidak ada MFA — credential leak = compromise total	High
C4	manakses query PDUT langsung → tightly coupled, blast radius luas	High
C5	RADIUS MySQL terpisah dengan format password berbeda → maintenance dual-system	Medium
C6	Library client <code>unila/sso</code> stagnan (30 commits, no release tag)	Medium
C7	Tidak ada audit log “siapa-login-kapan-dari-mana”	Medium
C8	CAS attribute hardcoded — sulit ganti format tanpa breaking semua aplikasi	Medium
C9	Tidak ada refresh token / session sliding — UX login berulang	Low
C10	Tidak ada social login / federasi (Google, eduroam)	Low

3. Tiga Opsi Upgrade

Opsi A — Apereo CAS Java Server

Pendekatan: Upgrade SSO Unila ke **Apereo CAS Server (Java)** — proyek yang sekarang aktif developed oleh Apereo Foundation. Banyak dipakai di higher-ed dunia (Stanford, MIT, Berkeley) dan Indonesia (UGM, ITB).

Komponen: - CAS Server Java (Spring Boot + Maven overlay) - LDAP / JDBC backend untuk user store - Optional: OAuth2/OIDC overlay module

Opsi B — Keycloak

Pendekatan: Deploy **Keycloak** sebagai IdP utama. Aplikasi baru pakai **OIDC native**. Aplikasi legacy `unila/sso` di-akomodasi via **lightweight CAS-compatible shim** atau community extension `keycloak-cas-services-provider`.

Komponen: - Keycloak (Quarkus, Java-based) - PostgreSQL untuk DB Keycloak - Optional: LDAP federation untuk sync ke FreeRADIUS

Opsi C — Custom Go IdP

Pendekatan: Bangun IdP sendiri di **Go + Fiber**, pakai library OIDC server seperti `zitadel/oidc`. Implement minimal CAS untuk backward-compat.

Komponen: - Service Go custom (backend/sso-service/) - PostgreSQL schema identity - Frontend admin (Next.js, di manajemen-akses)

4. Tabel Perbandingan Lengkap

4.0 Ringkasan 4 Opsi + Status Sekarang

Opsi	Stack	Pendekatan	Status Rekomendasi
Sekarang	phpCAS + custom PHP server	Legacy, security weak	✗ Tidak bisa dipertahankan (MD5 + SHA1)
A: Apereo CAS Java	Spring Boot CAS server	Incumbent path, native CAS	⚠ Solid tapi heavy stack
B: Keycloak	Quarkus, IdP modern	OIDC + SAML + CAS shim	✅ REKOMENDASI
C: Custom Go	Fiber + zitadel/oidc	Build sendiri	✗ Risk tinggi (security greenfield)
D: Entra ID Primary	Microsoft cloud	M365 native	✗ DITOLAK (lihat 4.6)

4.1 Perbandingan Umum

Aspek	Sekarang (CAS PHP)	Opsi A: Apereo CAS Java	Opsi B: Keycloak	Opsi C: Custom Go
Bahasa / Stack	PHP + phpCAS	Java (Spring Boot)	Java (Quarkus)	Go + Fiber
Lisensi	Apache 2.0 (phpCAS)	Apache 2.0	Apache 2.0	(self-owned)
Status maintainer	Library unila/sso stagnan, phpCAS maintenance mode	Aktif (release tiap kuartal)	Sangat aktif (release bulanan, Red Hat)	Internal Unila
Komunitas higher-ed	Banyak kampus pakai pattern serupa	Banyak (Stanford, MIT, UGM, ITB)	Sedang (banyak di Eropa)	Tidak ada referensi
Footprint memory produksi	~256 MB	~2 GB (JVM)	~1.5 GB (Quarkus lebih ringan dari Spring)	~256 MB
Footprint disk image	~150 MB	~600 MB	~500 MB	~80 MB
Cold start	<1 detik	30–60 detik	5–15 detik	<1 detik
Lengkungan belajar admin	Familiar (PHP)	Tinggi (Spring config)	Sedang (UI admin lengkap)	Tinggi (semua custom)

4.2 Dukungan Protokol & Standar

Protokol	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
CAS 1.0 / 2.0 / 3.0	✓	✓ (native)	⚠ (extension)	⚠ (build minimal shim)
OAuth 2.0	✗	✓ (overlay)	✓ (native)	✓ (via zitadel/oidc)
OpenID Connect (OIDC)	✗	✓ (overlay)	✓ (native, lengkap)	✓ (via zitadel/oidc)
SAML 2.0	✗	✓ (overlay)	✓ (native)	✗ (effort besar build)
SCIM 2.0 (provisioning)	✗	⚠ (limited)	✓ (native)	✗ (build)
LDAP (sebagai backend)	⚠	✓	✓	⚠ (custom)
LDAP (sebagai endpoint)	✗	⚠ (extension)	✓ (User Federation)	✗
WebAuthn / FIDO2 (passkey)	✗	⚠ (3rd party module)	✓ (native)	⚠ (library tersedia)
Social login (Google, Microsoft)	✗	✓ (overlay)	✓ (native, GUI config)	⚠ (build per-provider)

4.3 Keamanan

Aspek	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Password hashing	SHA1 (deprecated)	bcrypt / PBKDF2 / Argon2 (configurable)	Argon2id default + PBKDF2/bcrypt	Argon2id default
Password policy	Tidak ada	Configurable (length, complexity, history)	Configurable lengkap via UI	Custom
MFA TOTP	✗	✓ (overlay)	✓ (native, QR code generator)	Build (library pquerna/otp)
MFA WebAuthn/Passkey	✗	⚠	✓ (native)	Build
Rate limiting login attempt	✗ (atau minimal)	✓ (Throttle handlers)	✓ (brute force protection)	Build
Account lockout	✗	✓	✓	Build
Session security (HTTP-only, SameSite, CSRF)	Manual per app	✓	✓	Build
Token signing	Tidak relevan (ticket)	RSA / EC	RSA / EC (rotation built-in)	Build
Audit log	Tidak terstandar	✓	✓ (event listener API)	Build
Penetration testing track record	Tidak ada	Banyak (kampus + audit komersial)	Sangat banyak (Red Hat security team)	Tidak ada (greenfield)
CVE response cadence	Bergantung phpCAS upstream	<30 hari typical	<14 hari typical	Bergantung tim internal
Cryptographic library	mcrypt / openssl PHP	Java JCE	Java JCE + BouncyCastle	Go crypto/stdlib

4.4 Integrasi dengan Sistem Eksisting

Integrasi	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
App legacy (composer unila/sso)	Native	Native (no change)	Butuh CAS shim/extension	Butuh build CAS shim
App baru Next.js / Flutter	✗ (CAS tidak ramah SPA/mobile)	⚠️ (OIDC via overlay)	✅ (OIDC native, library standar)	✅
Query PDUT (source siapa user)	Direct query	User Federation (LDAP/JDBC)	User Federation (LDAP/JDBC/REST)	Custom adapter
Sync ke M365 / Google Workspace	Manual upload	SCIM outbound (limited)	✅ (built-in atau via identity-service)	Build
FreeRADIUS untuk WiFi	Custom HTTP API	LDAP / SQL pluggable	LDAP / REST native	Build REST
Mobile app (Flutter)	✗	OIDC overlay + PKCE	✅ (native PKCE)	Build PKCE
Audit log centralized	✗	Log4j → ELK	Event listener → Kafka/REST	Build
Eduroam-id federation	✗	SAML overlay	SAML native	Build SAML (effort besar)

4.5 Customization & Operasional

Aspek	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Branding login page	Hard-code template PHP	Theming via Spring view	✅ Theme via UI + Freemarker template	Custom build
Attribute mapper custom	Hardcoded di kode	Groovy script	✅ UI mapper + JavaScript	Custom code
Workflow login custom (e.g., consent screen)	Manual code per app	Authentication policy chain	✅ Authentication flow editor (drag-drop)	Custom code
Self-service password reset	❌ atau custom	✅ (overlay)	✅ (native)	Build
Admin UI untuk user management	Manual SQL	⚠️ Limited (CAS Mgmt Console terpisah)	✅ Sangat lengkap	Build dari nol
Realm / multi-tenant	❌	⚠️ Limited	✅ Native (mahasiswa, pegawai, eksternal di realm berbeda)	Build
High availability / clustering	Manual	Hazelcast clustering	✅ Built-in (Infinispan)	Build
Backup & restore	DB dump	DB dump + config files	DB dump + realm export JSON	DB dump
Observability (metrics, tracing)	Manual	Micrometer	Prometheus endpoint + OpenTelemetry	Build

4.6 RADIUS WiFi (Lihat juga Bagian 6)

Aspek RADIUS	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Path integrasi	Custom HTTP API → RADIUS MySQL terpisah	FreeRADIUS → LDAP (Apereo Person Directory)	FreeRADIUS → LDAP virtual (Keycloak) atau REST	FreeRADIUS → REST custom
EAP-TTLS-PAP support	⚠️ (dual hash)	✅	✅	✅
EAP-PEAP-MSCHAPv2 support	✅ (kalau dual-store NT-hash)	⚠️ (butuh NT-hash)	⚠️ (butuh dual-hash SPI)	Custom build
EAP-TLS (cert-based)	❌	✅ (CA built-in)	⚠️ (need external CA integration)	Build
Eduroam federation	❌	✅ (SAML + RADIUS proxy)	✅ (FreeRADIUS proxy + Keycloak SAML)	Build (besar)
VLAN/role attribute dari IdP	⚠️ (kalau ada di RADIUS MySQL)	✅ (LDAP attribute)	✅ (Keycloak user attribute)	Build

4.5 Referensi Implementasi Kampus Lain di Indonesia

Untuk benchmark, berikut pemetaan stack SSO kampus PTN besar (per riset 2026-05):

Kampus	URL SSO	Stack Backend	Protokol Utama	Catatan
ITS Surabaya	my.its.ac.id	OIDC native (DPTSI custom, likely Keycloak/IdentityServer-style)	OIDC + OAuth 2.0	Library client open-source: dptsi/php-openid-connect-client . Paling modern di antara kampus referensi.
IPB University	cas.ipb.ac.id/cas/login	Apereo CAS Java Server + OAuth overlay	CAS + OAuth 2.0	Sudah pakai server CAS proper (bukan client-only seperti Unila). Pattern persis = Opsi A.
Unila (sekarang)	sso.unila.ac.id	phpCAS client + custom PHP server	CAS only	Paling tertinggal — server-nya custom + library client legacy.
UGM, UI	(tidak publik)	Tidak terdokumentasi publik	—	Kemungkinan Keycloak/custom OIDC, perlu konfirmasi langsung.

Take-away: - ITS sudah migrasi ke **OIDC modern** — confirms pattern target Unila. - IPB pakai **Apereo CAS Java** — confirms Opsi A viable di konteks Indonesia. - Tidak ada kampus referensi yang pakai **Microsoft Entra langsung** sebagai primary IdP — ini bukan pola umum higher-ed.

4.6 Opsi D — Microsoft Entra ID sebagai Primary IdP (Dianalisis Ditolak)

Pertanyaan natural: kalau Unila sudah punya **M365 A1 for Education**, kenapa tidak pakai **Microsoft Entra ID** langsung sebagai IdP utama (gratis sudah include)?

Jawaban singkat: bisa secara teknis, tapi 3 deal-breaker membuatnya bukan kandidat.

Pro (Yang Bisa)

Fitur Entra ID Free / M365 A1	Status
Unlimited SSO ke SaaS apps (OIDC)	✓
MFA Authenticator app	✓
Self-service password reset (cloud user)	✓
Basic audit log	✓
Logo branding di login page	✓
SSO ke Teams/Outlook/OneDrive otomatis	✓
Compliance cert (FERPA, SOC 2, ISO 27001)	✓
Sudah gratis (no separate IdP cost)	✓

Con (Deal-Breaker)

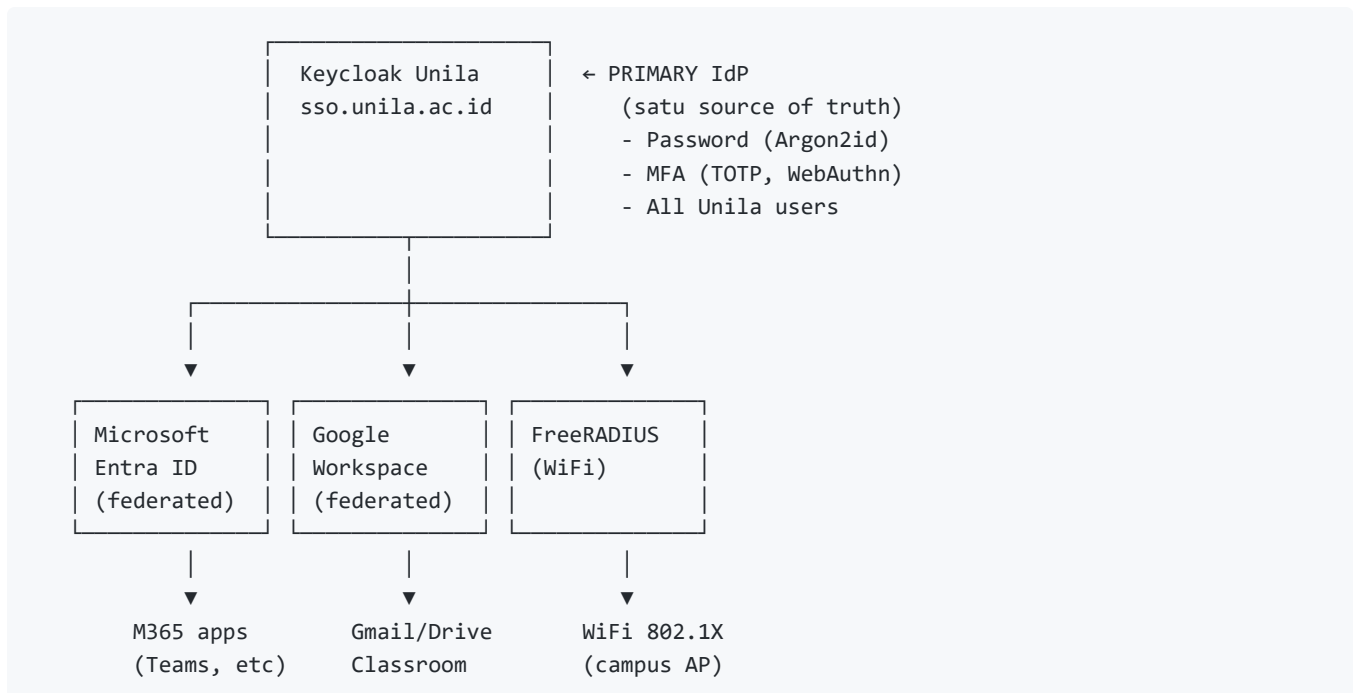
#	Masalah	Severity
D1	✗ CAS protocol tidak didukung Microsoft — semua app legacy composer require unila/sso mati. Tidak ada CAS shim resmi MS.	Critical
D2	✗ FreeRADIUS / WiFi integration tidak praktis — Entra ID bukan LDAP server. Microsoft NPS (alternatif) hanya cocok device joined-AD/Entra, mahasiswa pakai laptop/HP personal yang tidak joined.	Critical
D3	✗ Vendor lock-in total — semua identity di Microsoft. Kalau kebijakan/harga edu MS berubah, Unila terjebak. Strategis risky.	High
D4	Custom login UI hanya logo + warna — tidak bisa custom flow	High
D5	External user (alumni, tamu, vendor) batasi di free tier	High
D6	Conditional Access lengkap butuh P1/P2 — Unila ~50k user × \$9 = ~\$450k/bulan (tidak terjangkau)	High
D7	Custom attribute (NPM, prodi, jenjang) terbatas — extension attributes saja	Medium
D8	Eduroam-id federation rumit (perlu SAML config kompleks)	Medium
D9	Sync dari PDUT (PostgreSQL/SQL Server) → Entra tidak native — perlu custom Graph API integration	Medium
D10	Tidak ada kampus referensi yang pakai pattern ini sukses jangka panjang	Medium

Verdict Opsi D

Ditolak sebagai primary IdP. Tapi **tetap dipakai sebagai target downstream** lewat federation (lihat 4.7).

4.7 Federation Pattern (Pola yang Direkomendasikan)

Pola yang dipakai universitas modern (kemungkinan termasuk ITS):



Cara kerja:

1. Mahasiswa buka `outlook.unila.ac.id` (M365)
2. Microsoft kenali domain `@students.unila.ac.id` *federated to Keycloak* → redirect ke `sso.unila.ac.id`
3. User login di Keycloak (1× saja, branded myUnila biru)
4. Keycloak issue **SAML assertion** ke Microsoft → user masuk Outlook
5. Buka Gmail (Google Workspace) → flow sama, federated dari Keycloak
6. WiFi → 802.1X → FreeRADIUS REST ke Keycloak (juga 1× source)

One identity, one password, one MFA — semua via Keycloak.

Apa yang Diatur di Mana

Fungsi	Di Keycloak	Di Entra ID	Di Google Workspace
Password storage	✅ Master (Argon2id)	❌ Federated dari Keycloak	❌ Federated
MFA	✅ Master	❌ Skip (Keycloak handle)	❌ Skip
User attribute (NPM, prodi)	✅ Master	⚠️ Replica via SCIM/Graph	⚠️ Replica via Admin API
Group / role	✅ Master	Replica	Replica
Audit auth	✅ Master	✅ Hanya access M365	✅ Hanya access Google
Provisioning lifecycle	✅ Source	Replica (identity-service push)	Replica (identity-service push)

Konfigurasi Federation

Target	Cara
Microsoft 365	Verify custom domain <code>unila.ac.id</code> di Entra → Federation ke Keycloak via SAML 2.0 / WS-Fed → Entra tidak punya password store untuk user federated
Google Workspace	Admin Console → Security → SSO → SAML profile pointing ke Keycloak metadata URL
FreeRADIUS	<code>rlm_rest</code> module → REST endpoint Keycloak (Resource Owner Password Credentials grant)
App legacy	CAS shim service di depan Keycloak
App modern	OIDC native ke Keycloak

5. Perbandingan Aspek Keamanan

5.1 Skor Subjektif (1 = sangat lemah, 5 = sangat kuat)

Dimensi	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Password hashing strength	1	4	5	4 (depend on implementer)
MFA support	1	4	5	3
Brute force protection	1	4	5	3
Session security default	2	4	5	3
Audit logging	1	4	5	3
Vulnerability response (track record)	2	4	5	2 (no track record)
Token security (JWT signing, refresh)	N/A	4	5	3
Penetration testing maturity	1	5	5	1
Total	9 / 40	33 / 40	40 / 40	22 / 40

5.2 Vulnerability Track Record (Public Records)

Sistem	CVE 5 tahun terakhir	Severity tertinggi	Median time-to-patch
Apereo CAS	~8	High	~25 hari
Keycloak	~25	Critical (jarang)	~14 hari
phpCAS (current)	~6	Critical (RCE 2022 — CVE-2022-39369)	Slow upstream
Custom Go (hypothetical)	Tidak diketahui	Tidak diketahui	Bergantung tim internal

Catatan: Keycloak punya jumlah CVE lebih banyak karena coverage fitur lebih luas + transparansi disclosure tinggi (Red Hat SRT proaktif). Time-to-patch lebih cepat — indikator program security mature.

6. Integrasi RADIUS WiFi per Opsi

6.1 Konteks WiFi Kampus

- **Use case:** WiFi internal Unila (mahasiswa, dosen, staf, tamu).
- **SSID strategy:** Unila-Mahasiswa , Unila-Pegawai , Unila-Tamu , plus eduroam (jika join).
- **AP infrastruktur:** TBD — Cisco / Aruba / Mikrotik / Unifi (perlu konfirmasi).
- **Concurrent user peak:** estimasi 5k–10k saat jam padat.

6.2 Pilihan EAP Type

EAP Type	Mekanisme	Pro	Cons	Cocok untuk Argon2?
PAP (non-EAP)	Plaintext password	Sederhana	Tidak aman tanpa TLS	❌ (deprecated)
PEAP-MSCHAPv2	TLS tunnel + MSCHAPv2 inner	Native di Windows	Butuh NT-hash di server	❌ (perlu dual-store)
EAP-TTLS-PAP	TLS tunnel + PAP inner	Backend bisa verify Argon2	Sedikit konfigurasi manual di Android lama	✅ Recommended
EAP-TLS	Sertifikat per device	Paling aman, no password	Provisioning PKI berat	✅ N/A
EAP-PWD	Password-based, no MSCHAPv2	Modern, no MSCHAPv2 weakness	Support OS terbatas	✅

6.3 Aliran Detail RADIUS per Opsi

Opsi A: FreeRADIUS → Apereo CAS LDAP/JDBC

```
[Device WiFi]
| 802.1X EAP-TTLS-PAP
▼
[Access Point]
| RADIUS request
▼
[FreeRADIUS 3.x]
| unwrap TLS → plaintext (NPM, password)
| rlm_ldap query
▼
[Apereo CAS Person Directory / LDAP backend]
| verify password (Argon2 / PBKDF2 via Apereo PasswordEncoder)
| return user attributes: role, vlan_id
▼
[FreeRADIUS] map LDAP attribute → RADIUS attribute:
- Tunnel-Type=VLAN
- Tunnel-Medium-Type=IEEE-802
- Tunnel-Private-Group-ID=100 (VLAN mahasiswa)
▼
[Access-Accept] → AP assign VLAN
```

Kelebihan: LDAP adalah pattern matang FreeRADIUS, banyak dokumentasi. **Kekurangan:** Setup Apereo Person Directory butuh effort; CAS punya internal user store yang tidak otomatis expose LDAP.

Opsi B: FreeRADIUS → Keycloak (LDAP virtual atau REST)

Variasi 1: via REST (paling fleksibel)

```
[Device WiFi]
| EAP-TTLS-PAP
▼
[FreeRADIUS]
| rlm_rest POST
| URL: https://sso.unila.ac.id/realms/unila/protocol/openid-connect/token
| body: grant_type=password&client_id=radius&username=NPM&password=...
▼
[Keycloak]
| verify password (Argon2id)
| check MFA (kalau enable untuk role WiFi – biasanya tidak)
| check account status (a_aktif, suspended)
| return access_token + claims
▼
[FreeRADIUS] parse JWT claims:
- role: "mahasiswa" → VLAN 100
- role: "pegawai" → VLAN 200
- role: "tamu" → VLAN 300
▼
[Access-Accept + VLAN attribute]
```

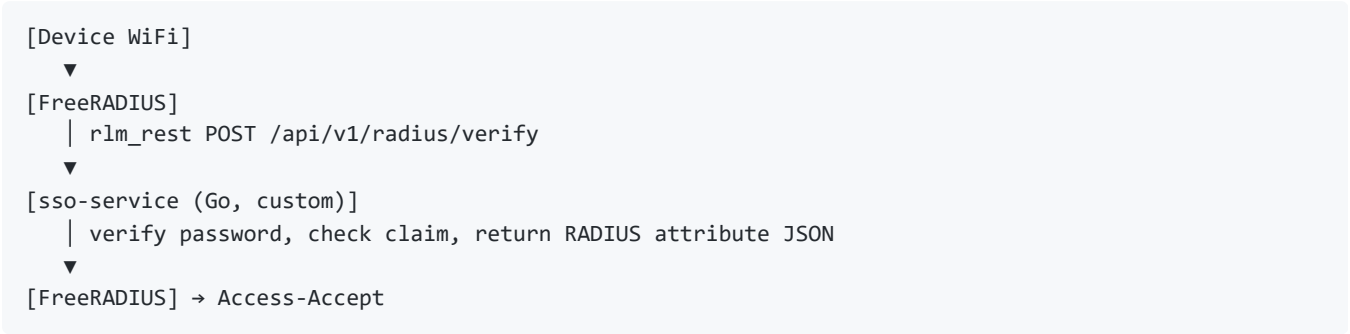
Variasi 2: via LDAP (Keycloak sebagai LDAP server)

Keycloak tidak native expose LDAP, tapi bisa pakai pattern **OpenLDAP slave** yang di-sync dari Keycloak via SCIM atau script. FreeRADIUS query OpenLDAP slave.

Kelebihan REST: real-time, no sync, full claim available. **Kelebihan LDAP:** RADIUS performance lebih tinggi (LDAP connection pool), familiar pattern.

Rekomendasi: REST untuk simplicity, evaluate LDAP kalau performance jadi bottleneck (>2k req/sec).

Opsi C: FreeRADIUS → Custom Go IdP via REST



Kelebihan: full control, performant. **Kekurangan:** semua harus dibangun, termasuk endpoint khusus RADIUS, attribute mapping, dst.

6.4 Tabel Ringkas Integrasi RADIUS

Aspek	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Backend RADIUS pakai	MySQL custom	LDAP (Apereo Person)	REST atau LDAP virtual	REST custom
Setup complexity (1–5)	3 (legacy yang sudah jalan)	4 (LDAP overlay)	2 (REST straightforward)	5 (build from scratch)
Password hash compatibility	Native (kalau format konsisten)	Configurable via Apereo encoder	Configurable via SPI	Full control
Argon2id support	✗	✓	✓	✓
MFA untuk WiFi (opsional)	✗	⚠ Limited	✓ via Keycloak flow	Build
VLAN attribute dari claim	Manual	LDAP attribute mapping	✓ Native attribute mapper	Build
Eduroam-id home server	✗	✓ (RADIUS proxy)	✓ (FreeRADIUS proxy + Keycloak SAML)	Build
Failover / HA	Manual	Hazelcast	Infinispan	Build
Connection pooling	Native MySQL	LDAP pool	HTTP pool atau LDAP pool	HTTP pool
Performance (estimasi req/sec)	500–1000	1000–2000 (LDAP)	1500–3000 (REST + cache)	Bergantung implementasi

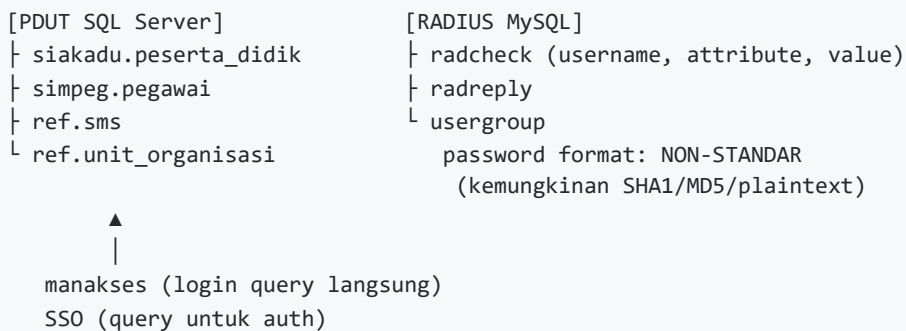
6.5 Strategi WiFi Multi-SSID (target arsitektur)

SSID	EAP Type	VLAN	Akses
Unila-Mahasiswa	EAP-TTLS-PAP	100	Internet + intranet akademik
Unila-Pegawai	EAP-TTLS-PAP	200	Internet + intranet penuh
Unila-Tamu	Open + Captive Portal	300	Internet rate-limited
eduroam (jika join)	EAP-TTLS-PAP, realm @unila.ac.id	100/200	Sesuai peran

VLAN attribute dikembalikan FreeRADIUS berdasarkan claim `role` dari IdP.

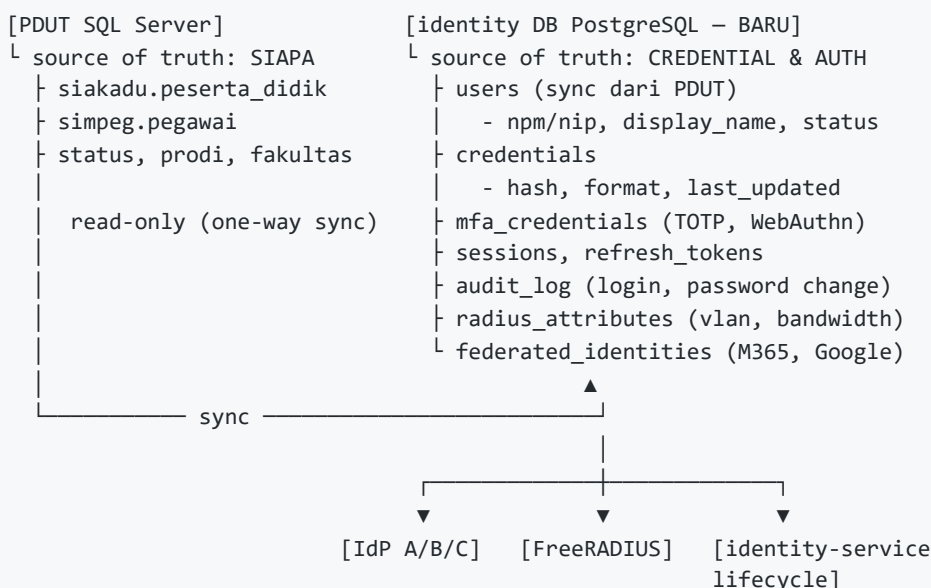
7. Arsitektur Database

7.1 Skema Saat Ini



Masalah: - 2 DB dengan format password berbeda — harus sinkron manual saat user ganti password. - PDUT terkait erat dengan auth — coupling tinggi. - Tidak ada DB khusus untuk session, MFA, audit auth.

7.2 Skema Target (untuk semua opsi A/B/C)



7.3 Pisah DB: Argumen per Aspek

Aspek	Shared (PDUT)	Pisah (identity DB)	Pemenang
Blast radius	Auth down = SIAKAD down	Auth down = SIAKAD aman	Pisah
Schema evolution (MFA, WebAuthn, sessions)	Harus modifikasi PDUT shared	Bebas	Pisah
Backup / RTO	Mengikuti PDUT (besar, slow restore)	Bisa lebih agresif (small, fast)	Pisah
Performance burst (5k connect bersamaan)	PDUT bisa kewalahan	Identity DB di-tune untuk burst	Pisah
Compliance (UU PDP, data minimization)	PDUT terekspos ke network RADIUS	Hanya identity DB	Pisah
Operational complexity	1 DB	2 DB + sync	Shared (sedikit)
Konsistensi atribut user (nama, prodi)	Native (1 source)	Butuh sync	Shared
Cost storage	Re-use	+marginal (~5 GB)	Shared (sedikit)

Skor: 6–2 untuk pisah DB. Rekomendasi: pisah.

7.4 Sync PDUT → Identity DB

Tipe sync	Trigger	Latency	Implementation
Event-driven (write-through)	Saat ada <code>INSERT/UPDATE peserta_didik</code> di PDUT	< 1 menit	Trigger DB → message queue → identity-service
Nightly reconciliation	Scheduled 02:00	24 jam	Full diff PDUT vs identity DB
On-demand	Admin klik "Sync user X" di UI	Real-time	API call ke identity-service
Webhook si-registrasi	NPM generated	< 5 menit	(sudah dibahas di doc Identity Lifecycle)

Pattern: kombinasi event-driven (real-time) + nightly (safety net).

8. Strategi Migrasi Password Lama

8.1 Format Password yang Ada Saat Ini (Terkonfirmasi)

Format	Konteks	Status	Reversibility	Catatan Keamanan
MD5(password) tanpa salt	RADIUS MySQL eksisting (untuk auth WiFi)	Confirmed	One-way	Critical — rainbow table publik tersedia; password lemah bisa di-crack dalam detik
SHA1(YYYYMMDD + NPM)	SSO Unila (dari SsoService.php di si-registrasi)	Confirmed (kode source)	One-way	Critical — deterministic (tgl lahir + NPM diketahui = password ter-tebak)
Mix lain (kemungkinan minoritas)	User lama yang sudah pernah ganti password	Belum diverifikasi	—	Perlu sampling per-record

Implikasi MD5 tanpa salt: - Attacker dengan dump DB RADIUS MySQL bisa **brute-force semua password lemah <30 detik** pakai hashcat/john + GPU. - Password lemah seperti `123456`, `password`, `unila123` ter-crack instan via rainbow table. - Tidak ada salt → **dua user dengan password sama = hash sama** (data leak privacy).

Implikasi RADIUS MySQL juga dipakai aplikasi web: - Username yang sama login WiFi & web pakai password yang sama → kalau salah satu compromised, akses ke yang lain. - Migrasi harus **simultan untuk WiFi + web** — tidak bisa rolling per-channel.

8.2 Pattern Rehash-on-Login (Recommended)

Berlaku untuk **semua opsi A/B/C** dengan cara implementasi berbeda:

Konsep: 1. Import semua user as-is dengan **field tambahan** `password_format`. 2. IdP punya **multi-format verifier** — verify input dengan format yang sesuai. 3. Saat login sukses, **auto-rehash ke Argon2id**.

```

function verify_login(npm, plaintext_input):
    user = find_user(npm)
    stored_hash = user.password_hash
    format = user.password_format

    match format:
        case "sha1-unila":
            tgl_lahir = user.tgl_lahir // ambil dari PDUT
            expected = sha1(format_date(tgl_lahir) + npm)
            valid = (expected == stored_hash)

        case "md5":
            valid = (md5(plaintext_input) == stored_hash)

        case "plaintext":
            valid = (plaintext_input == stored_hash)

        case "argon2id":
            valid = argon2.verify(plaintext_input, stored_hash)

    if valid and format != "argon2id":
        user.password_hash = argon2.hash(plaintext_input)
        user.password_format = "argon2id"
        save(user)
        audit_log("password_rehashed", user.id)

    return valid

```

8.3 Implementasi per Opsi

Opsi	Cara Implementasi
A: Apereo CAS	Custom PasswordEncoder di Java, plug ke authentication handler
B: Keycloak	Custom PasswordHashProvider SPI (Java), deploy sebagai JAR ke providers/
C: Custom Go	Function di service code (paling sederhana, tinggal kode)

8.4 Timeline Sunset Format Lama

Bulan	Aksi
0	Migrasi import + deploy verifier multi-format
1–6	Pengguna aktif login otomatis ter-rehash
6	Email reminder ke user yang belum login pasca-migrasi
12	Force password reset untuk yang belum re-hash
18	Disable SPI legacy, hanya Argon2id

8.5 MFA — Migrasi dari MyUnila Existing ke Keycloak

Kondisi Saat Ini (Confirmed)

MyUnila sudah punya implementasi MFA TOTP:

Komponen	Detail
Library	<code>pragmarx/google2fa-laravel ^2.3</code>
Backend	<code>backend/auth-service/</code> Laravel — <code>MfaService.php</code> , <code>mfa_tokens</code> table
Frontend	<code>frontend/src/app/portal/settings/page.tsx</code> (UI setup/disable)
Auth flow	<code>AuthContext.tsx</code> cek <code>mfa_required</code> flag dari login response
Endpoint	<code>POST /auth/setup-mfa</code> , <code>/auth/verify-mfa</code> , <code>/auth/disable-mfa</code>
Compatible	Google Authenticator, Authy, MS Authenticator (standar TOTP RFC 6238)
Scope	Hanya proteksi login MyUnila — bukan untuk app lain
Status	Opt-in per-user (tidak wajib)

Masalah MFA di Application Layer

#	Masalah	Dampak
1	MFA hanya proteksi MyUnila — SIAKAD, LMS, email, WiFi tetap tanpa MFA	Inkonsistensi cross-system
2	Pasca-Keycloak: MyUnila tidak validate password sendiri lagi (redirect OIDC)	MFA logic Laravel jadi dead code
3	Kalau MFA tetap di MyUnila + tambah di Keycloak = user setup 2×	UX buruk
4	Maintenance dobel (lockout policy, backup code, bug fix)	Beban dev

Prinsip Federated SSO

MFA harus di IdP layer, bukan app layer. Setiap app yang re-implement MFA = anti-pattern. Pasca-migrasi: MFA cukup satu kali di Keycloak, semua app dapat manfaat-nya via SSO.

Migrasi Data MFA (Tanpa User Setup Ulang)

TOTP secret existing bisa dipindah ke Keycloak tanpa user re-scan QR code — Authenticator app pakai secret yang sama, kode 6-digit tetap valid.

Sumber	Target	Cara
<code>auth_service.mfa_tokens.secret</code> (base32)	Keycloak <code>user_credential</code> <code>type= otp</code>	One-time script: Keycloak Admin API <code>POST /admin/realms/unila/users/{id}/credentials</code>
<code>mfa_tokens.enabled = true</code>	Keycloak credential ter-aktif	Migration script otomatis
Backup codes existing	Generate ulang di Keycloak	User notify: "backup codes lama tidak berlaku, ambil baru di <code>sso.unila.ac.id/account</code> "
<code>mfa_tokens.created_at</code>	Audit log entry	Catat di Keycloak event log

Kebijakan MFA Target Pasca-Migrasi

Role	MFA Wajib	Method	Catatan
Admin sistem (UPA TIK)	✅ Wajib	TOTP + WebAuthn	High-privilege
Dosen / Staf	✅ Wajib	TOTP	Standar profesional
Pimpinan (Rektor, WR, Dekan)	✅ Wajib	WebAuthn / Passkey	Target phishing high-value
Mahasiswa Fase 1	⚠️ Opsional	TOTP	UX gradual
Mahasiswa Fase 2	✅ Wajib	TOTP	Phased rollout
Mahasiswa baru (PMB)	✅ Auto-enroll	TOTP saat first login	Best practice dari awal
Alumni	✅ Wajib	TOTP	Akun lama = target phishing
WiFi (RADIUS)	❌ Tidak MFA	—	UX device 802.1X tidak support; password Argon2id cukup

Step-Up Authentication untuk Action Sensitif

Setelah MFA di-IdP-layer, app masih bisa **request re-auth dengan MFA wajib** untuk operasi krusial via OIDC standard:

```
// MyUnila trigger re-auth dengan MFA wajib (mis: approve perubahan IPK)
loginWithAuthorizationRequest({
  acr_values: "mfa",      // standar OIDC ACR claim
  prompt: "login",       // force re-auth
  max_age: 0,
});
```

Keycloak terima request → minta TOTP code lagi → balik ke MyUnila dengan claim `acr=mfa` di `id_token`. MyUnila verify claim sebelum eksekusi action.

Keuntungan: logic MFA tetap satu tempat, tapi step-up untuk sensitive operation tetap bisa di-enforce per-app.

Yang Dihapus Pasca-Migrasi MFA

File / Komponen di MyUnila	Aksi
composer require pragmarx/google2fa-laravel	Hapus dari composer.json
mfa_tokens table	Archive lalu drop
MfaService.php , bagian MFA di TokenService.php	Hapus
Endpoint /auth/setup-mfa , /auth/verify-mfa , /auth/disable-mfa	Hapus dari routes/api.php
/portal/settings UI MFA section	Ganti jadi tombol “Kelola MFA” → redirect ke Keycloak Account Console
AuthContext.tsx mfa_required flow	Hapus (Keycloak handle dalam redirect-nya)

9. Estimasi Biaya & Effort per Opsi

9.1 Biaya Lisensi (semua Rp 0)

Komponen	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
Lisensi software	Rp 0 (Apache)	Rp 0 (Apache)	Rp 0 (Apache)	Rp 0 (in-house)
Lisensi support (opsional)	—	Apereo Foundation membership ~\$2k/yr	Red Hat SSO subscription opsional (~\$50k/yr)	—
Lisensi cloud	—	—	—	—

Catatan: Unila tidak perlu paid support — community + dokumentasi cukup untuk skala kampus.

9.2 Infrastruktur

Item	Sekarang	A: Apereo CAS	B: Keycloak	C: Custom Go
RAM service	256 MB	2 GB	1.5 GB	256 MB
RAM DB	(shared PDUT)	+1 GB PostgreSQL	+1 GB PostgreSQL	+1 GB PostgreSQL
Disk image	150 MB	600 MB	500 MB	80 MB
Disk DB	(shared)	+5 GB	+5 GB	+5 GB
VM tambahan	0	1 (atau co-host)	1 (atau co-host)	0 (di VM existing)
FreeRADIUS VM	(existing)	4 GB RAM	4 GB RAM	4 GB RAM

Hardware cost: marginal — semua bisa di-host di VM existing Unila.

9.3 Engineering Effort

Fase	A: Apereo CAS	B: Keycloak	C: Custom Go
Spike & sandbox	1 minggu	1 minggu	2 minggu
Deploy + config initial	2 minggu	1 minggu	—
Migration password SPI	1 minggu	1 minggu	1 minggu
Import user DB	1 minggu	1 minggu	1 minggu
CAS legacy compat	Native (0)	1–2 minggu (shim)	2 minggu (build shim)
OIDC integration MyUnila	1 minggu	1 minggu	2 minggu (build OIDC)
FreeRADIUS REST/LDAP	2 minggu	1 minggu	2 minggu
Admin UI	2 minggu (Mgmt Console)	0 (built-in)	4 minggu (build dari nol)
MFA setup	1 minggu	1 minggu	2 minggu
Audit log + monitoring	1 minggu	1 minggu	2 minggu
Pilot & cutover	2 minggu	2 minggu	3 minggu
Total wall-time	~15 minggu	~11–12 minggu	~21 minggu
Total person-effort (rough)	~80 hari	~60 hari	~120 hari

9.4 Skala TCO 3 Tahun

Komponen	A	B	C
Build cost (hari kerja)	80	60	120
Maintenance / bulan (hari)	4	3	8
3-tahun maintenance	144	108	288
Total 3 tahun (hari kerja)	224	168	408

Opsi B paling efisien dari sisi engineering effort.

10. Rekomendasi Final & Justifikasi

10.1 Rekomendasi: Opsi B (Keycloak) + Federation ke Entra+Google + FreeRADIUS REST + Pisah DB `identity` + MFA di IdP

10.2 Justifikasi

1. **Security paling matang** — skor 40/40, vulnerability response <14 hari, password hashing modern default, MFA + WebAuthn native.
2. **Effort engineering paling rendah** — 60 hari vs 80 (A) atau 120 (C). UI admin sudah ada, tidak perlu bangun.
3. **Customization mudah** — theming via UI, authentication flow editor drag-drop, attribute mapper, tanpa rebuild Java.
4. **Standar protokol terluas** — OIDC + SAML + LDAP federation native; CAS via shim untuk legacy.
5. **Path migrasi mulus** — Keycloak custom SPI mendukung rehash-on-login; legacy `composer unila/sso` tetap kompatibel.
6. **RADIUS integration cleanest** — REST endpoint langsung dari FreeRADIUS, support EAP-TTLS-PAP dengan Argon2id.
7. **Roadmap masa depan** — mudah tambah social login, eduroam, federation antar kampus, MFA passkey.
8. **Komunitas raksasa** — Stack Overflow ratusan ribu Q&A, Red Hat dukungan, banyak tutorial.

10.3 Yang Perlu Diwaspadai

Risk Keycloak	Mitigasi
Java stack (skill team)	UI admin meminimalkan code Java. SPI custom (password verifier) ~200 baris Java. Tim Go bisa belajar atau outsource SPI saja.
CAS bukan first-class	Shim ~500 baris, atau pakai community provider <code>keycloak-cas-services-provider</code> (Apache 2.0).
Resource lebih berat dari Go	1.5 GB RAM masih masuk akal untuk core auth infra.
Upgrade major version (Keycloak versi $N \rightarrow N+1$)	Test di staging dulu, ada migration tool. Cadence release kuartal.

10.4 Decision Log

#	Keputusan	Pilihan	Alasan
1	IdP utama	Keycloak	Security matang, effort rendah, UI lengkap
2	DB credential	Pisah dari PDUT — identity PostgreSQL	Isolasi blast radius, schema evolution bebas
3	Sumber atribut user	PDUT (sync one-way ke identity DB)	PDUT tetap authoritative untuk SIAPA
4	RADIUS WiFi protocol	EAP-TTLS-PAP	Compatible dengan Argon2id, UX sama dengan PEAP
5	RADIUS backend	FreeRADIUS → Keycloak REST	Real-time, no sync, claim langsung tersedia
6	Legacy CAS compat	Shim service di depan Keycloak	Aplikasi composer unila/sso tetap jalan tanpa modifikasi
7	Password migration	Rehash-on-login + custom SPI	Zero-downtime, transparent ke user
8	MFA strategy	Wajib untuk dosen/staf/admin, opsional mahasiswa di phase 1	Balance security vs UX
9	Login MyUnila man-akses	Switch ke OIDC ke Keycloak	Tidak lagi query PDUT langsung untuk auth
10	Branded login page	Theme Keycloak myUnila biru	Konsistensi visual

11. Open Questions untuk Stakeholder

#	Pertanyaan	Pihak yang Diharapkan Menjawab
Q1	Stack Java (Keycloak) dapat diterima atau strict harus Go/PHP?	UPA TIK + tim infra
Q2	~~Format password persis di RADIUS MySQL & SSO Unila sekarang?~~ Terjawab: MD5 tanpa salt (RADIUS MySQL) + SHA1(YYYYMMDD+NPM) (SSO Unila) . Apakah ada user lama dengan format lain?	Admin SSO eksisting
Q3	Inventory aplikasi pakai composer require unila/sso — perlu daftar semua app legacy yang harus tetap kompatibel pasca-migrasi. (Konfirmasi awal: ada beberapa app + auth WiFi pakai DB yang sama)	Tim Dev MyUnila
Q4	WiFi controller vendor Cisco / Aruba / Mikrotik / Unifi? (vendor-specific RADIUS attribute mapping) — butuh ngobrol tim infra	Tim Infra Jaringan
Q5	Concurrent WiFi session peak di jam padat? — butuh ngobrol tim infra	Tim Infra Jaringan
Q6	Unila sudah join eduroam-id atau belum? Kalau belum, ada rencana?	UPA TIK + Kerjasama
Q7	Device WiFi target — masih support Windows 7 / Android lama ? (memengaruhi pilihan EAP) — butuh ngobrol tim infra	Tim Infra Jaringan
Q8	MFA wajib untuk role mana ? Dosen+staf wajib, mahasiswa opsional di phase 1 — setuju?	UPA TIK + WR Akademik
Q9	Domain SSO — pakai <code>sso.unila.ac.id</code> existing atau migrate ke <code>auth.unila.ac.id</code> ?	UPA TIK
Q10	Owner & operasional Keycloak pasca-deploy: tim yang sama dengan MyUnila atau dedicated SSO admin?	Pimpinan UPA TIK
Q11	Budget paid support Red Hat / Apereo membership dipertimbangkan, atau community-only?	Pimpinan UPA TIK
Q12	Co-host VM Keycloak dengan service MyUnila atau VM dedicated?	Tim Infra
Q13	Password reset flow — self-service via email pribadi (yang ada di si-registrasi) atau via admin TIK?	UPA TIK
Q14	Captive portal Unila-Tamu — pakai Keycloak realm khusus, integrasi dengan AP, atau standalone tool?	Tim Infra Jaringan
Q15	Cutover timing — rolling per-app atau big-bang? Periode aman (di luar UTS/UAS/PMB)	UPA TIK + Akademik

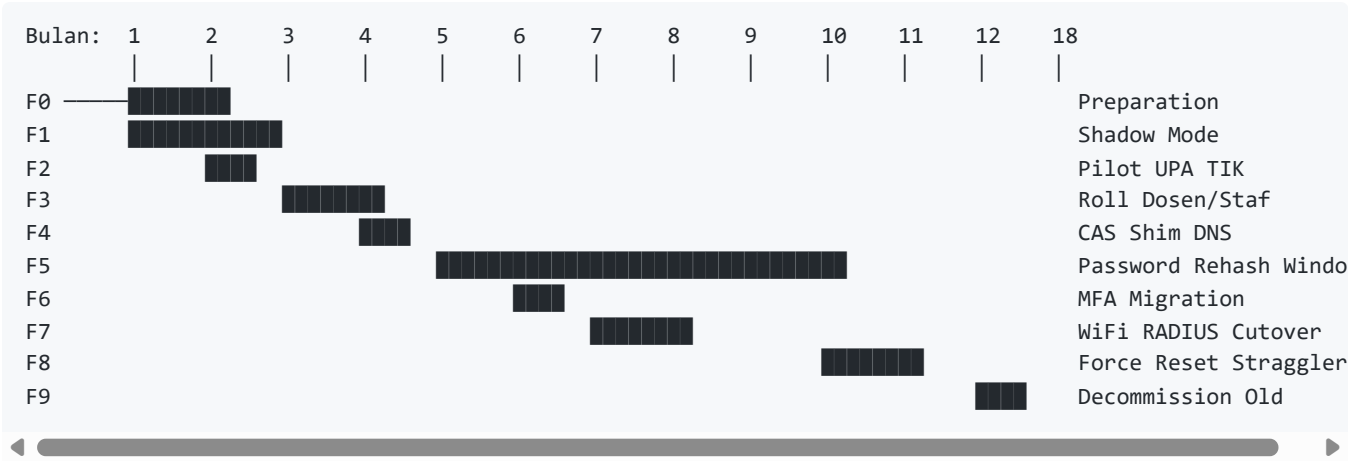
12. Tahapan Migrasi Aman

Prinsip utama: Migrasi melibatkan ~50.000 user (mahasiswa + dosen + staf + alumni) dan WiFi kampus. **Zero-downtime, zero-surprise, fully reversible per fase.** Tidak ada cutover big-bang.

12.1 Prinsip Operasional

#	Prinsip	Konsekuensi Operasional
1	Zero downtime	Selama transisi, sistem lama tetap jalan parallel dengan Keycloak
2	Rollback per fase	Setiap fase ada DNS/feature-flag switch back; tidak ada keputusan yang tidak bisa dibatalkan dalam 1× klik
3	Komunikasi proaktif	Setiap fase didahului email + banner + WhatsApp broadcast minimal 7 hari sebelum
4	Pilot dulu	Tidak ada role yang langsung mass-migrate; selalu pilot 30–100 user dulu
5	Helpdesk siap	Setiap fase didahului training admin TIK & helpdesk (Tiket Q&A standar disiapkan)
6	Self-service	Reset password / unlock / re-enroll MFA — semuanya self-service via portal, bukan tergantung admin
7	Observability	Dashboard real-time success/fail rate login; alert otomatis kalau ada anomali
8	Periode aman	Tidak dilakukan di tengah UTS/UAS, masa pendaftaran KRS, atau PMB pagi hari

12.2 Diagram Timeline (9 Fase)



12.3 Fase Detail

Fase 0 — Preparation (1–2 bulan, NO USER IMPACT)

Aktivitas	Output
Deploy Keycloak di VM staging	Keycloak instance running
Theme branding myUnila biru	Login page branded
Build schema <code>identity</code> PostgreSQL	DB ready
Build custom Keycloak SPI password verifier (MD5, SHA1-unila)	JAR deployed
Build CAS shim service	Service running
Import sample 100 user dari PDUT (test)	Verified
FreeRADIUS REST integration testing di test SSID	Authenticated
Build admin tooling (migration script, monitoring)	Ready
Training tim TIK	Done

Rollback strategy: trivial — staging only, tidak menyentuh user.

Komunikasi: internal saja (tim TIK).

Fase 1 — Shadow Mode (2–4 minggu, NO USER IMPACT)

Keycloak berjalan parallel dengan SSO lama. **Dual-write password change:**

Aktivitas
Saat user ganti password di SSO lama → trigger sync ke identity DB (Argon2id) via webhook
Saat user enable MFA → secret di-sync ke Keycloak credential
Reconciliation nightly: diff antara user di SSO lama vs identity DB
Test login Keycloak untuk admin TIK saja (private)

Tujuan: Pastikan data sync sempurna sebelum cutover satu user pun.

Rollback strategy: matikan webhook sync, identity DB tetap ada tapi unused.

Komunikasi: internal tim TIK + pimpinan.

Fase 2 — Pilot UPA TIK (1 minggu, 30–50 user)

User pilot: **staf UPA TIK** (yang paling siap troubleshoot kalau ada masalah).

Hari	Aktivitas
-7	Pengumuman ke staf UPA TIK
-3	Email berisi: link reset password sementara, panduan setup MFA, link helpdesk
0	Switch login route untuk grup ini ke Keycloak (via feature flag user-group)
0-7	Daily standup tim TIK: review login attempt, error, feedback
+7	Decision: lanjut atau rollback

Metric sukses: - $\geq 95\%$ pilot user login sukses dalam 24 jam - ≤ 5 helpdesk ticket per user terkait login - 0 ticket “tidak bisa kerja sama sekali”

Rollback strategy: flip feature flag balik ke SSO lama. Identity DB tetap (untuk lanjutkan nanti).

Komunikasi: email + WhatsApp group UPA TIK.

Fase 3 — Rolling Migration per Kohort (4–6 minggu, ~50k user)

Migrate per kohort dengan jadwal aman:

Minggu	Kohort	Estimasi User
W1	Pimpinan + Dekan (mereka harus paling siap karena visibilitas)	~50
W2	Dosen FT + FMIPA (faculty IT-savvy)	~500
W3	Dosen semua fakultas	~1.500
W4	Staf administrasi	~1.000
W5	Mahasiswa angkatan baru (kalau ada periode PMB sedang berlangsung — mereka lebih mudah karena belum punya legacy account)	~5.000
W6–W10	Mahasiswa existing per angkatan (dari yang terbaru ke yang lama)	~40.000

Per kohort, alur 1 minggu:

| H-7 | Email blast: “Akun Anda akan dipindah ke sso.unila.ac.id pada [tanggal]” + link FAQ + video tutorial |
 | H-3 | WhatsApp/SMS reminder | | H-1 | Banner di MyUnila + email reminder akhir | | H | Switch feature flag — login untuk kohort ini lewat Keycloak | | H+1..7 | Monitoring intensif, helpdesk siaga | | H+7 | Lock-out login lewat SSO lama untuk kohort ini |

Critical: Mahasiswa yang sedang **UTS/UAS** atau **registrasi KRS** TIDAK dimigrasi minggu itu — tunggu periode aman.

Rollback strategy per kohort: feature flag balik, user kembali lewat SSO lama (data identity DB tetap synced).

Komunikasi multi-channel: - Email blast (template myUnila biru) - Banner di MyUnila dashboard - WhatsApp blast (untuk yang nomornya ada di SIAKADU) - Video tutorial 2 menit (di YouTube channel

Fase 4 — CAS Shim Activation untuk Legacy Apps (1 minggu)

Setelah semua user sudah migrasi:

Aktivitas
Switch DNS <code>sso.unila.ac.id</code> → CAS shim service di depan Keycloak
SSO lama tetap online di IP lain sebagai fallback (selama 30 hari)
Test semua app <code>composer require unila/sso</code> masih jalan
Monitor usage SSO lama → expect: 0 traffic pasca-DNS switch

Untuk user: tidak ada perubahan UX — login URL sama, atribut user yang dikembalikan sama (`username` , `email` , `nm_pengguna` , `a_aktif`).

Rollback strategy: switch DNS balik ke SSO lama (TTL 60 detik). Apps tetap jalan.

Fase 5 — Password Rehash Window (6 bulan, transparent ke user)

Selama 6 bulan pasca-Fase 4, setiap login sukses pakai MD5/SHA1 lama → auto-rehash ke Argon2id (lihat 8.2).

Bulan ke-	Aksi
1	60–70% user aktif sudah ter-rehash
3	85–90% sudah ter-rehash
6	95%+ sudah ter-rehash
6	Email reminder ke user yang belum login pasca-migrasi

Untuk user: tidak terasa — login dengan password yang sama.

Fase 6 — MFA Migration (1 minggu, user dengan MFA aktif saja)

Migrate MFA secret existing dari MyUnila ke Keycloak (lihat 8.5).

Hari	Aktivitas
H-7	Email ke user yang MFA enabled: “MFA Anda akan dipindah, Anda tidak perlu setup ulang”
H-3	Script export <code>mfa_tokens.secret</code> dari MyUnila DB
H	Script import ke Keycloak credential via Admin API (bulk)
H+1	User login → diminta TOTP — kode dari Authenticator app yang sama tetap valid
H+7	Validate: % user yang sukses input MFA pasca-migrasi
H+30	Deprecate endpoint <code>/auth/verify-mfa</code> di MyUnila

Untuk user: transparan — tidak perlu re-scan QR code di Authenticator app.

Rollback strategy: kalau ada bulk failure, restore MyUnila MFA endpoint, request user verify lewat MyUnila lama selama 24 jam debugging.

Fase 7 — WiFi RADIUS Cutover (2 minggu)

Migrate auth WiFi dari RADIUS MySQL lama ke FreeRADIUS REST → Keycloak.

Minggu	Aktivitas
W1	Setup FreeRADIUS baru di VM dedicated, point ke Keycloak
W1	Test SSID Unila-Test di 1 gedung — staff UPA TIK pakai 1 minggu
W2 D1	Switch SSID Unila-Pegawai ke FreeRADIUS baru
W2 D3	Switch SSID Unila-Mahasiswa (jam istirahat sore, low traffic)
W2 D5	Old RADIUS MySQL: monitor traffic → expect 0
W2 D7	Keep old RADIUS standby selama 30 hari (rollback option)

Untuk user WiFi: - **Connection profile yang sama** (username NPM + password yang sudah dipakai di MyUnila) - Device tidak perlu re-pair / re-input — kalau “Remember credentials” sudah enable, transparent
- Kalau device pernah save password lama (MD5), perlu update password sekali (atau force reset di Fase 8)

Rollback strategy: SSID config di AP switch balik ke RADIUS MySQL lama (10 menit operation).

Komunikasi WiFi: poster di kampus, banner di MyUnila, instagram TIK.

Fase 8 — Force Reset Stragglers (1–2 minggu, 5–10% user)

User yang tidak login 12 bulan pasca-migrasi → masih punya MD5/SHA1 hash.

Aktivitas
Query: WHERE password_format != 'argon2id' AND last_login_at < NOW() - 12 months
Email + SMS: “Akun Anda perlu reset password sebelum [tanggal], login berikutnya akan otomatis force change”
Hari H: flag user → required_action = UPDATE_PASSWORD di Keycloak
Saat user login berikutnya: dipaksa ganti password (hashed Argon2id otomatis)
Setelah 30 hari: user yang belum reset → akun di-suspend, perlu kontak helpdesk

Untuk user aktif: tidak terdampak (sudah pakai Argon2id).

Untuk user inaktif: tindakan ringan (cuma reset password sekali).

Fase 9 — Decommission Old SSO (1 minggu, 6+ bulan pasca-Fase 4)

Aktivitas
Verify: 6 bulan zero traffic ke SSO lama
Backup full data SSO lama (untuk arsip 7 tahun)
Decommission VM SSO lama
Decommission RADIUS MySQL VM lama
Drop <code>mfa_tokens</code> table di MyUnila (sudah archive)
Remove dependency <code>pragmarx/google2fa-laravel</code>
Update dokumentasi internal: SSO Unila = Keycloak only

No user impact (server sudah lama tidak menerima traffic).

12.4 Komunikasi & Change Management

Channel Komunikasi per Fase

Channel	Fase 0–1	Fase 2 (Pilot)	Fase 3 (Roll)	Fase 4–5	Fase 6 (MFA)	Fase 7 (WiFi)
Internal email TIK	✓	✓	✓	✓	✓	✓
Email blast civitas	—	✓ (pilot only)	✓ (per kohort)	—	✓ MFA user	—
Banner MyUnila	—	—	✓	✓	✓	✓
WhatsApp blast	—	—	✓	—	✓	✓
Instagram/Twitter TIK	—	—	✓	—	—	✓
Poster di kampus	—	—	✓ (mahasiswa)	—	—	✓
Video tutorial YouTube	—	✓	✓	—	✓	✓
FAQ page	✓ live	✓ update	✓ update	✓	✓ update	✓ update
Helpdesk extended hours	—	✓	✓	✓	✓	✓

Template Komunikasi (Contoh)

Email H-7 ke Kohort Mahasiswa:

Subject: Akun SSO Anda akan dipindah ke sistem baru pada [DD/MM/YYYY]

Halo [Nama],

Universitas Lampung sedang meningkatkan keamanan sistem SSO.
Mulai [DD/MM/YYYY], login ke MyUnila/SIKAD/email akan menggunakan halaman login baru di sso.unila.ac.id.

Yang berubah:

- ✓ Tampilan login lebih modern (warna biru myUnila)
- ✓ Sistem lebih aman (Argon2id, MFA tersedia)
- ✓ One login untuk M365 + Google + WiFi

Yang TIDAK berubah:

- NPM dan password Anda tetap sama
- Tidak perlu setup ulang
- Tidak perlu install aplikasi baru

Bantuan:

- FAQ: unila.ac.id/sso-baru
- Video tutorial: [youtu.be/\[ID\]](https://youtu.be/[ID])
- Helpdesk: 0721-xxx atau helpdesk@unila.ac.id

Terima kasih,

UPA TIK Universitas Lampung

12.5 Risk Register Tahapan Migrasi

Risiko	Mitigasi
User panik karena tampilan login baru	Tampilan branded myUnila biru identik dengan tema MyUnila; banner edukasi "ini sistem baru kami"
Massive helpdesk ticket di hari migrasi	Extended hours; staf tambahan; FAQ page lengkap dengan screenshot
WiFi tiba-tiba tidak konek device user	Old RADIUS standby 30 hari; SSID test dulu; rollback 10 menit
MFA secret hilang saat migrasi → user terkunci	Backup <code>mfa_tokens</code> sebelum migrate; self-service MFA reset di Keycloak Account Console
Apps legacy yang tidak terdokumentasi tiba-tiba broken	Pre-migration: inventory exhaustive semua subdomain <code>*.unila.ac.id</code> ; staging test
User lupa password setelah lama tidak login	Self-service reset; email pribadi (yang ada di si-registrasi) jadi recovery
Pimpinan complain karena UX berbeda	Fase 3 Week 1 = pimpinan dulu, dengan onboarding personal
Eduroam-id federation rusak	Federation config di-test di staging dulu; rollback ke standalone WiFi sementara
Reconciliation drift PDUT vs identity DB	Nightly job + manual review dashboard di manajemen-akses
Migration script gagal di tengah	Idempotent, checkpoint per-user, dapat resume dari point of failure

12.6 Definition of Success per Fase

Fase	Success Criteria
F0	Keycloak running, theme done, SPI working in staging
F1	Sync drift < 0.1% selama 14 hari berturut-turut
F2	≥ 95% pilot user login sukses, ≤ 5 ticket per user
F3 (per kohort)	≥ 98% kohort login sukses dalam 7 hari, ≤ 0.5% helpdesk ticket rate
F4	100% app legacy tetap functional pasca-DNS switch
F5	≥ 95% user aktif sudah rehash dalam 6 bulan
F6	≥ 99% MFA user sukses login pakai TOTP code yang sama
F7	WiFi success rate ≥ 99.5%, ≤ 5 ticket per hari pasca-cutover
F8	100% user yang belum reset → otomatis di-reset atau di-suspend
F9	SSO lama decommissioned bersih, no data loss

12.7 Critical Calendar (Periode Aman vs Hindari)

Periode	Status	Aksi
Awal semester (KRS, pembayaran)	✗ Hindari	Pause migration
UTS / UAS	✗ Hindari	Pause migration
PMB / Registrasi mahasiswa baru	⚠ Hati-hati	Migrate per-individu OK, hindari mass
Wisuda	✗ Hindari (email pengumuman wisuda)	Pause
Libur akademik	✓ Ideal	Bulk migration
Antara semester (kuliah berjalan normal)	✓ Aman	Default window

Rekomendasi window: mulai pasca-wisuda + libur akademik (mis: Agustus, Februari) — peak attention TIK, low academic disruption.

13. Appendix A: Protocol Glossary

Daftar singkat protokol identity yang sering tertukar:

A.1 Web SSO Protocols

Protokol	Format Token	Kasus Pakai	Status
CAS 1.0/2.0/3.0	Service Ticket (opaque)	Higher-ed legacy	Legacy, masih dipakai
SAML 2.0	Assertion (XML signed)	Enterprise SSO, federation, eduroam, Shibboleth	Standar B2B/edu aktif
OAuth 2.0	Access Token	API authorization	Standar API
OpenID Connect (OIDC)	ID Token (JWT)	Modern web/mobile auth	Standar modern
WS-Federation	SAML token via WS-*	Microsoft legacy (ADFS)	Deprecated

OAuth 2.0 ≠ OIDC: OAuth = authorization, OIDC = authentication. Untuk login user, **selalu OIDC**.

A.2 Directory / Backend Store Protocols

Protokol	Fungsi
LDAP v3	Query directory tree (user, group, OU)
LDAPS / StartTLS	LDAP over TLS (wajib production)
Active Directory	LDAP + Kerberos + DNS (Microsoft)
SCIM 2.0	REST API untuk provisioning user (CRUD)
JDBC/SQL	Query DB langsung (legacy pattern)

LDAP bukan SSO protocol — dia *directory*. Sering disangka SSO karena dipakai backend banyak SSO server.

A.3 Network-Level Auth

Protokol	Layer	Kasus Pakai
RADIUS	UDP 1812/1813	WiFi (802.1X), VPN
RadSec	TCP 2083 (TLS)	RADIUS modern (eduroam)
Kerberos	UDP/TCP 88	Intra-network AD domain
EAP	Inside 802.1X	Method untuk RADIUS WiFi: EAP-TLS, PEAP, EAP-TTLS, EAP-PWD

A.4 Token Formats

Format	Dipakai oleh
JWT	OIDC, OAuth2, banyak custom — <code>header.payload.signature</code> base64
SAML Assertion	SAML 2.0, Shibboleth — XML signed
CAS Service Ticket	CAS — opaque string, validate via callback
Opaque Token	OAuth2 opaque mode — random string, validate via introspection

A.5 MFA / Strong Auth

Protokol	RFC / Spec	UX
TOTP	RFC 6238	Authenticator app, kode 6-digit refresh 30 detik
HOTP	RFC 4226	Counter-based, jarang dipakai
WebAuthn / FIDO2	W3C, FIDO Alliance	Passkey — biometrik / hardware key
U2F	FIDO U2F legacy	Hardware key 2FA, sudah digantikan WebAuthn
Push notification	Vendor-specific	Tap “approve” di HP
SMS / Email OTP	Vendor	Tidak direkomendasikan (SIM swap attack)

A.6 Federation Standards

Standar	Kasus Pakai di Kampus
SAML 2.0 Federation	Keycloak ↔ M365, ↔ Google Workspace
Shibboleth	SAML implementation khusus higher-ed (eduroam, eduGAIN)
OIDC Federation	OIDF spec — federasi antar OIDC provider (lebih baru)

A.7 Pemetaan Protokol di Unila Target

Skenario	Protokol
Login app web modern (MyUnila Next.js)	OIDC ke Keycloak
Login app mobile (Flutter)	OIDC + PKCE ke Keycloak
Login app legacy (composer unila/sso)	CAS via shim → Keycloak
Login M365 (Teams, Outlook)	SAML 2.0 federation Keycloak → Entra
Login Google Workspace	SAML 2.0 federation Keycloak → Google
WiFi 802.1X	RADIUS + EAP-TTLS-PAP → FreeRADIUS → Keycloak
Eduroam roaming	RADIUS proxy + SAML federation
Sync user M365 / Google / HRIS	SCIM 2.0 atau Microsoft Graph / Google Admin SDK
MFA primary	TOTP + WebAuthn
Token IdP → app	JWT (signed RS256/ES256)

Versi dokumen: v2 (brainstorm + safe migration plan) **Update terakhir:** 13 Mei 2026 **Catatan:** Dokumen ini bersifat brainstorm dengan tahapan migrasi aman. Setelah jawaban Open Questions (Bagian 11) terkumpul, akan disusun **Business Process Document final** dengan implementation plan detail (mirip docs/identity-lifecycle/business-process.md).